TP RSA Sécurité et Pratiques

Arthur Walter

Master 2 Data Science · Sécurité des systèmes d'information

Enseignant Abdeljalil ABBAS-TURKI · 16 octobre 2026

Objet du travail

Ce rapport accompagne la réalisation des trois parties du TP RSA. Il présente les calculs, le code, les résultats obtenus et leur interprétation. Il sert aussi de guide pour reproduire le travail dans VS Code et comprendre la logique de chaque étape.

Bob envoie un message à Alice. Il le chiffre avec la clé publique d'Alice, puis signe son empreinte avec sa propre clé privée. Alice déchiffre et vérifie la signature. L'exercice ajoute ensuite une simulation de canal auxiliaire, le calcul CRT et un bourrage pédagogique par blocs.

Les sources prioritaires sont TP_RSA_Enonce.pdf et RSA_B_A.py dans TP_secu_1/sources. Exp_CRT_RSA.pdf, dans ce même dossier, précise la recombinaison CRT. Le programme conserve les facteurs premiers et les exposants publics du professeur.

Résultats essentiels

Les trois parties fonctionnelles sont implémentées. Quinze tests automatisés contrôlent le calcul, les conversions, les traces, le CRT et le bourrage. Trois tests supplémentaires vérifient l'analyse de certificats X.509. La première CI a validé les calculs sous Python 3.10 et 3.13.

Le certificat présenté par link.springer.com a été observé avec validation TLS le 16 octobre 2026 à 16:02:25 UTC depuis GitHub Actions. Le certificat PEM et les champs extraits sont conservés dans resultats.

Parcours de lecture

Les pages 2 à 6 expliquent l'exécution et la partie 1. Les pages 7 à 9 traitent les traces et le CRT. Les pages 10 et 11 présentent le bourrage et la restauration des messages. Les pages 12 à 14 couvrent le certificat, les tests, les limites et les livrables.

L'implémentation est pédagogique : petites clés, calculs Python variables dans le temps et signature brute non standard. Les résultats fonctionnels ne constituent pas une preuve de sécurité.

Exécuter le TP dans VS Code

Préparer Python

Dans VS Code, ouvrir Terminal > New Terminal et exécuter python --version. Si cette commande échoue, essayer py --version. Le programme complet nécessite Python 3.10 ou plus récent, notamment pour le comptage de bits avec int.bit_count.

Si Python est absent, l'installer depuis https://www.python.org/downloads/windows/. Avec l'installateur classique, activer Add python.exe to PATH si l'option est proposée. Rouvrir VS Code après installation.

Installer l'extension Python publiée par Microsoft, identifiant ms-python.python. Ouvrir directement le dossier TP_secu_1 dans VS Code, puis Ctrl+Maj+P > Python: Select Interpreter et sélectionner Python.

Lancer les démonstrations

Le programme de calcul n'a aucune dépendance externe à installer. Les imports de Walter_Arthur_util.py sont locaux ; conserver les deux fichiers Python ensemble.

```
python Walter_Arthur_main.py --test
python Walter_Arthur_main.py --partie toutes --message "Bonjour Alice"
python Walter_Arthur_main.py --partie 2 --message "Bonjour"
python Walter_Arthur_main.py --partie 3 --message "Un message long avec accents é"
```

L'option --partie 1 impose 25 octets UTF-8. L'option toutes exécute aussi cette partie et conserve donc cette limite. Pour un long message, choisir --partie 3. Sans --message, le terminal demande le texte à saisir.

L'option --sortie resultats/mon_execution.json enregistre les résultats en plus de les afficher. Les configurations F5 fournies dans .vscode permettent de lancer la démonstration, un message long ou les tests. Un point d'arrêt dans la marge permet de suivre les variables avec F10.

Résoudre les erreurs courantes

Si le terminal affiche No such file or directory, vérifier le dossier avec Get-Location et dir. Si l'interpréteur n'est pas détecté, relancer Python: Select Interpreter. Si le programme attend, saisir le message dans le terminal et appuyer sur Entrée.

Contrairement au build manuel habituel d'un projet .NET, aucune commande de compilation n'est nécessaire ici : l'interpréteur exécute le .py. Le relevé de certificat utilise séparément cryptography ; son installation n'est nécessaire que pour relancer cet outil optionnel.

Partie 1 Représentation et clés RSA

Comprendre les variables

Un nombre premier est un entier supérieur à 1 divisible seulement par 1 et lui-même. Alice et Bob possèdent chacun deux premiers p et q. Leur produit n est le module RSA. La clé publique est formée de n et e ; la clé privée utilise d.

$$n = p \times q$$

$$\lambda(n) = PPCM(p - 1, q - 1)$$

$$e \times d \equiv 1 \ (mod \ \lambda(n))$$

Le symbole ≡ signifie que les deux nombres ont le même reste. Par exemple, 17 modulo 5 vaut 2, car 17 = 3 × 5 + 2. En Python, % calcule le reste et // effectue la division entière.

Paramètre	Alice	Bob
Exposant public e	17	23
Taille du module n	203 bits	206 bits
Bloc brut garanti	25 octets	25 octets

Le squelette appelle phia et phib les résultats d'un calcul par PPCM. Il s'agit de λ(n), la fonction de Carmichael pour ces modules, et non de φ(n). Pour deux premiers distincts, φ(n) est le produit (p − 1) × (q − 1). Le code adapté utilise lambda_a et lambda_b pour clarifier cette distinction.

Transformer le texte sans perdre ses octets

Le message est encodé en UTF-8 puis converti en entier big-endian. Le premier octet est le plus significatif. Pour AB, les octets sont 65 et 66 : l'entier vaut 65 × 256 + 66 = 16 706.

```
donnees = secret.encode("utf-8")
num_sec = int.from_bytes(donnees, "big")
texte = entier.to_bytes(taille_octets, "big").decode("utf-8")
```

La longueur en octets accompagne l'envoi de la partie 1. Elle permet de conserver les zéros initiaux et le message vide. Un caractère accentué peut utiliser plusieurs octets : é compte pour un caractère mais deux octets UTF-8.

Les conversions historiques du squelette ont été remplacées ensemble. Le bloc triple guillemets initial non refermé a été supprimé et les deux fonctions vides complétées. Les fichiers originaux sont conservés dans le dossier sources.

Partie 1 Exponentiation modulaire rapide

Réponse à la question 1a

La fonction home_mod_expnoent calcule une puissance modulo n par carrés et multiplications. Calculer d'abord toute la puissance produirait un entier énorme. Réduire chaque produit modulo n conserve le bon résultat final avec des intermédiaires plus petits.

```
def home_mod_expnoent(x, y, n):
    if n <= 0 or y < 0:
        raise ValueError("Il faut n > 0 et y >= 0.")
    resultat = 1 % n
    base = x % n
    exposant = y
    while exposant > 0:
        if exposant % 2 == 1:
            resultat = (resultat * base) % n
        base = (base * base) % n
        exposant //= 2
    return resultat
```

resultat commence à 1, l'élément neutre de la multiplication. À chaque tour, un exposant impair sélectionne la base courante. Le carré suivant représente une puissance deux fois plus grande ; la division entière de l'exposant fait passer au bit suivant.

Le nombre de tours est proportionnel au nombre de bits de y. La boucle examine les bits du poids faible au poids fort. Ce choix explique l'ordre des traces de la partie 2.

Exemple de calcul

Pour calculer 3 puissance 13 modulo 17, décomposer 13 = 8 + 4 + 1, soit 1101 en binaire. La fonction sélectionne les puissances correspondant aux trois bits à 1.

Exposant avant tour	Base avant tour	Résultat après sélection
13 impair	3	3
6 pair	9	3
3 impair	13	5
1 impair	16	12

Le résultat est 12. Le test compare 300 cas reproductibles à pow(x, y, n). Cette fonction native sert uniquement de référence de test ; elle ne remplace pas l'algorithme demandé dans les calculs du TP.

Partie 1 Euclide étendu

Réponse à la question 1b

home_ext_euclide(y, b) conserve l'ordre du squelette : y est le module et b le nombre à inverser. Pour calculer la clé privée, on appelle donc home_ext_euclide(lambda_a, ea), puis l'équivalent pour Bob.

```
def home_ext_euclide(y, b):
    if y <= 1:
        raise ValueError("Le module doit dépasser 1.")
    ancien_r, r = y, b % y
    ancien_t, t = 0, 1
    while r:
        quotient = ancien_r // r
        ancien_r, r = r, ancien_r - quotient * r
        ancien_t, t = t, ancien_t - quotient * t
    if ancien_r != 1:
        raise ValueError("Pas d'inverse.")
    return ancien_t % y
```

Cet extrait abrège les messages d'erreur du fichier source ; les opérations sont identiques. Euclide calcule le PGCD par divisions successives. La version étendue conserve le coefficient de b dans une combinaison de y et b.

$$PGCD(y,\ b) = s \times y + t \times b$$

Si le PGCD vaut 1, le terme multiple de y disparaît modulo y. Il reste t × b congru à 1 : t est l'inverse recherché. Le modulo final donne un représentant positif. Si le PGCD n'est pas 1, aucun inverse n'existe et le programme lève une erreur.

Vérification numérique

Pour le couple classique module 3120 et exposant 17, le résultat est 2753.

$$17 \times 2753 = 46801 = 15 \times 3120 + 1$$

Le reste vaut 1. Les tests vérifient aussi les inverses pour des modules de 2 à 69, les refus lorsqu'il n'existe pas d'inverse et les deux égalités ea × da % lambda_a = 1 et eb × db % lambda_b = 1.

La clé privée d ne doit pas être confondue avec un mot de passe. C'est un entier calculé à partir de l'exposant public et des facteurs privés du module. Dans cet exercice, les facteurs sont explicitement fournis pour permettre l'apprentissage.

Partie 1 Échange et SHA 256

Réponse à la question 2

Bob chiffre l'entier du message avec la clé publique d'Alice. Alice utilise ensuite son exposant privé pour retrouver cet entier. Bob signe séparément l'empreinte avec sa clé privée ; Alice la vérifie avec la clé publique de Bob, supposée fiable.

Action	Clé utilisée
Chiffrer pour Alice	Publique Alice
Déchiffrer chez Alice	Privée Alice
Signer chez Bob	Privée Bob
Vérifier chez Alice	Publique Bob

L'exécution sur Bonjour Alice retrouve exactement ce texte et retourne signature_valide = true. Le message comporte 13 octets ; sa représentation entière est 526354504649867183149086837232 5. Le chiffré et les deux signatures figurent dans resultats/execution_complete.json.

Réponse à la question 3a

MD5 est remplacé par hashlib.sha256(message.encode("utf-8")).digest(). L'empreinte contient toujours 32 octets, même pour un long texte. Sa forme hexadécimale contient 64 caractères. La conversion intermédiaire binascii.b2a_uu est retirée.

Le module de Bob ne fait que 206 bits. L'entier correspondant aux 256 bits du SHA-256 peut le dépasser. Pour conserver l'empreinte entière et les clés fournies, cette adaptation la découpe en blocs de 25 puis 7 octets, signés séparément. Ce choix pédagogique est explicite ; le sujet ne prescrit pas cette solution. Ce n'est ni RSA-PSS ni une signature standard sécurisée.

Réponse à la question 3b

Pour tout contenu de L octets, la condition suffisante est 256 puissance L − 1 < na. Avec le module d'Alice de 203 bits, 25 octets occupent 200 bits et tiennent toujours. Un bloc arbitraire de 26 octets peut occuper 208 bits et dépasser na.

La nouvelle limite uniforme est donc 25 octets UTF-8 : 25 lettres ASCII, ou 12 caractères é suivis d'une lettre ASCII. Treize caractères é atteignent 26 octets et sont refusés. Certains contenus particuliers de 26 octets pourraient tenir numériquement, mais la politique retenue les refuse.

SHA-256 n'augmente pas la capacité du bloc RSA. Le passage de la restriction de 10 caractères à 25 octets vient de la validation et de l'encodage corrigés. Le traitement de textes plus longs est apporté par la partie 3.

Partie 2 Traces et canal auxiliaire

Réponse à la question 1

home_mod_expnoent_trace reprend le calcul rapide et enregistre chaque bit examiné. Un bit à 1 déclenche la multiplication conditionnelle resultat × base. Chaque tour effectue également un carré. Le résultat Trace contient le résultat numérique, le nombre de multiplications conditionnelles, le nombre de carrés et la suite des bits observés.

```
bit = exposant % 2
bits.append(bit)
if bit:
    resultat = resultat * base % n
base = base * base % n
exposant //= 2
```

Pour 3 puissance 13 modulo 17, le programme retourne 12, compte trois multiplications conditionnelles et quatre carrés. L'ordre chronologique des bits est 1, 0, 1, 1, du poids faible au poids fort. La représentation binaire habituelle s'obtient en inversant cet ordre : 1101.

Ce que révèle la trace

Un simple total de multiplications conditionnelles révèle le poids de Hamming de l'exposant, c'est-à-dire son nombre de bits à 1. Il ne révèle pas leur position : 11 et 13 ont tous les deux trois bits à 1 et une longueur de quatre bits, mais leurs représentations sont 1011 et 1101.

Si un observateur distingue idéalement chaque carré et chaque multiplication dans le temps, il peut associer les multiplications aux positions et reconstituer les bits dans le modèle simplifié de l'exercice. Une mesure réelle comporte du bruit et dépend du matériel et de l'implémentation.

Dans nos essais, les déchiffrements des messages 1, 42, 513 et 123456789 ont tous 85 multiplications conditionnelles et 198 carrés. La clé privée d'Alice est identique dans ces quatre opérations, donc son poids et sa longueur ne changent pas avec le message.

Limite de la simulation

Les événements sont enregistrés volontairement à l'intérieur de la fonction. Ils simulent des pics observables ; ils ne sont pas une mesure physique d'énergie, de cache ou d'émanation. Le code n'affirme pas avoir récupéré une clé à partir d'un chronométrage réel.

La fonction n'est pas à temps constant. Le branchement sur les bits secrets et les opérations sur grands entiers Python peuvent produire des variations. Le CRT de la section suivante améliore le calcul sans supprimer automatiquement cette fuite.

Partie 2 Déchiffrement par CRT

Réponse à la question 3

Calculer directement le déchiffrement revient à travailler modulo n, le produit des deux premiers. Le CRT permet de calculer séparément modulo p et modulo q, avec des exposants réduits, puis de recombiner les résultats.

$$dp = d \mod (p - 1)$$

$$dq = d \mod (q - 1)$$

$$mp = c^{dp} \mod p$$

$$mq = c^{dq} \mod q$$

$$q_inv = \text{inverse de } q \text{ modulo } p$$

$$h = ((mp - mq) \times q_inv) \mod p$$

$$m = (mq + h \times q) \mod n$$

q_inv est calculé par notre fonction d'Euclide étendu. La fonction home_preparer_crt calcule une seule fois dp, dq et q_inv ; home_rsa_crt les réutilise. La comparaison temporelle ne recalcule donc pas ces paramètres à chaque déchiffrement.

La réduction d'exposant repose sur le petit théorème de Fermat pour les résidus non nuls modulo les premiers. Le code traite aussi explicitement un chiffré divisible par p ou q : le résidu doit rester nul, même si l'exposant réduit est zéro.

Reproduire l'exemple du professeur

Paramètre	Valeur	Paramètre	Valeur
p	137	q	131
n	17947	c	8363
d	11787	q_inv	114
dp	91	dq	87
mp	102	mq	120

On obtient h = ((102 − 120) × 114) modulo 137 = 3, puis m = 120 + 3 × 131 = 513. Le rechiffrement avec e = 3 donne 513 puissance 3 modulo 17947 = 8363 : le message original est bien retrouvé.

Les tests comparent aussi le CRT à l'exponentiation directe pour les deux clés du TP, sur des cas limites et 80 chiffrés pseudoaléatoires par clé. Les valeurs 0, 1, p, q et n − 1 sont incluses.

Partie 2 Bonus et interprétation

Comparaison des temps

La fonction home_comparer_temps utilise perf_counter_ns. Pour chaque message, elle effectue cinq séries de trente déchiffrements et conserve la médiane du temps moyen par opération. L'ordre direct puis CRT est alterné entre les séries pour limiter un biais d'ordre.

Les résultats suivants proviennent de resultats/execution_complete.json, sous Python 3.12.14 sur Linux. Les valeurs sont des observations locales, pas des performances garanties.

Message entier	Direct en ns	CRT en ns	Rapport direct sur CRT
1	62661	43763	1,432
42	361520	182812	1,978
513	267391	246892	1,083
123456789	156573	112352	1,394

Un rapport supérieur à 1 correspond ici à un CRT plus rapide. Ces petites clés et le coût de l'interpréteur rendent les temps sensibles à la charge de la machine. Une nouvelle exécution donnera probablement d'autres valeurs. Le message 1 produit des résidus particuliers très peu coûteux.

Le total de multiplications conditionnelles reste 85 pour les quatre calculs directs. Les variations de temps ne s'expliquent donc pas uniquement par ce total : la valeur des opérandes et les coûts d'exécution interviennent aussi. Ces mesures ne démontrent pas une récupération de clé.

Le CRT réduit il la surface d'attaque

Le sujet invite à expliquer pourquoi le CRT réduit la surface d'attaque. La réponse doit être nuancée : il réduit le coût et souvent la durée du calcul, mais cela ne suffit pas à établir une réduction générale du risque de canal auxiliaire.

Les deux exponentiations CRT manipulent encore des exposants secrets dp et dq. Une implémentation vulnérable peut laisser fuiter ces valeurs. Le CRT peut aussi exposer aux attaques par faute : un calcul erroné sur une branche peut, dans certains scénarios, aider à retrouver un facteur du module.

Les protections usuelles comprennent des primitives conçues pour un temps constant, l'aveuglement et la vérification du résultat. Dans cet exercice, le calcul direct sert de référence pour les tests ; cette comparaison n'est pas présentée comme une protection complète intégrée à un protocole de production.

Le résultat pédagogique est double : le CRT restitue le même message et peut accélérer le déchiffrement ; l'optimisation mathématique ne remplace pas une implémentation résistante aux attaques.

Partie 3 Découpage et bourrage

Réponse à la question 1 étapes a à f

La partie 1 limite le message à un seul entier RSA. Pour transmettre un texte plus long, le programme encode d'abord tout le texte en UTF-8, le découpe en morceaux de j octets, puis bourre et chiffre chaque morceau séparément.

Le module d'Alice occupe 203 bits. Sa taille arrondie en octets vaut k = 26. La consigne impose j inférieur ou égal à k divisé par deux. On choisit donc j maximal = 13 octets.

Chaque bloc est construit sous la forme suivante. Le symbole || signifie concaténation d'octets.

$$EM = 00 \,||\, 02 \,||\, PS \,||\, 00 \,||\, morceau$$

Élément	Taille avec un morceau de 13 octets
Préfixe 00 02	2 octets
PS non nul	10 octets
Séparateur 00	1 octet
Morceau du message	13 octets
Total	26 octets

La taille de PS vaut k − j − 3. Au dernier bloc, si le morceau est plus court, PS est plus long afin de conserver 26 octets au total. La contrainte supplémentaire PS d'au moins 8 octets est également vérifiée. Notre maximum de 13 octets est plus restrictif que la limite k − 11 de PKCS#1 v1.5, conformément au sujet.

Générer des octets non nuls

Chaque octet de PS est obtenu par secrets.randbelow(255) + 1, dans l'ensemble de 1 à 255. C'est le même domaine de valeurs que alea % 255 + 1 demandé par l'énoncé. Le tirage direct évite un éventuel biais lié au modulo d'une source mal choisie.

```
ps = bytes(secrets.randbelow(255) + 1
           for _ in range(k - len(bloc) - 3))
em = b"\x00\x02" + ps + b"\x00" + bloc
```

EM est converti en entier big-endian, puis chiffré avec la clé publique d'Alice. Le premier octet nul borne sa valeur à moins de 256 puissance 25, donc en dessous du module utilisé. Le code vérifie aussi explicitement cette condition avant de chiffrer.

Partie 3 Restaurer et contrôler le message

Réponse à la question 1 étapes g à i

Alice déchiffre chaque entier par CRT, puis reconstruit exactement k = 26 octets avec to_bytes. Cette taille fixe est essentielle : sans elle, la conversion en entier aurait supprimé le premier octet nul du préfixe.

home_debourrer vérifie la longueur, le préfixe 00 02, l'existence du premier séparateur nul après le préfixe, au moins huit octets de PS et le respect de j maximal = 13. Les octets précédant ce premier séparateur sont nécessairement non nuls.

Le morceau suivant le séparateur est récupéré. Tous les morceaux sont concaténés en octets, puis le décodage UTF-8 est effectué une seule fois à la fin. Ainsi, un caractère accentué ou un emoji coupé entre deux blocs n'est pas perdu.

Résultat sur un message long

Le message testé est : Bonjour Alice ! Ce message dépasse vingt-cinq octets et contient des accents é et un emoji 😀.

Mesure	Résultat
Longueur UTF-8	98 octets
Taille maximale d'un morceau	13 octets
Nombre de blocs chiffrés	8
Texte restitué	Identique à l'original
Signature du message complet	Valide
CRT et calcul direct	Résultats identiques

Les détails sont enregistrés dans resultats/message_long.json. Un message vide est représenté par un bloc bourré contenant zéro octet utile. Les zéros appartenant au message sont conservés : ils se trouvent après le séparateur.

Le bourrage varie aléatoirement à chaque exécution, donc deux chiffrements du même texte donnent normalement des entiers différents. La signature brute reste déterministe dans cette adaptation. Les tests vérifient aussi le rejet d'une signature manquante et la détection de blocs permutés ou supprimés dans des cas concrets.

Erreurs et padding oracle

Les erreurs de longueur, de préfixe, de séparation et de décodage utilisent un message générique. Cela évite de multiplier les catégories visibles, mais ne rend pas le code Python à temps constant. Un serveur exposant la validité du bourrage, y compris par son temps de réponse, pourrait créer un oracle.

Le programme est une simulation locale, sans service réseau de déchiffrement. Ce format est inspiré de PKCS#1 v1.5 ; il ne constitue pas RSA-OAEP et ne transforme pas l'échange en protocole sécurisé.

Partie 3 Étude du certificat Springer

Réponse à la question 2

Le certificat a été relevé par une connexion TLS à link.springer.com:443 avec SNI link.springer.com, contrôle de la chaîne et du nom d'hôte actifs. La collecte a eu lieu depuis le runner GitHub Actions le 16 octobre 2026 à 16:02:25 UTC, soit 18:02:25 à Paris. Le chemin /journal/43684 n'intervient pas dans la sélection du certificat, qui précède la requête HTTP.

Champ observé	Valeur
Sujet	CN=*.springer.com
Émetteur	CN=YR2, O=Let's Encrypt, C=US
Clé publique du site	RSA de 2048 bits
Exposant public	65537, soit 0x10001
Signature du certificat	sha256WithRSAEncryption
OID de signature	1.2.840.113549.1.1.11
Hash de cette signature	SHA-256
Encodage de cette signature	RSASSA-PKCS1-v1_5
Début de validité UTC	23 septembre 2026 à 11:18:57
Fin de validité UTC	22 décembre 2026 à 11:18:56

Les données sont dans resultats/certificat_springer.json et le certificat public dans certificat_springer.pem. L'empreinte SHA-256 du certificat, distincte du nom du hash utilisé par sa signature, est :

`27d5c26b2047ca2b459030694a9fed4e68157c22b802be2ffb231ee09d327350`

La correspondance de ces champs avec le PEM téléchargé a été vérifiée localement. La preuve d'exécution est le workflow 38928738912 et son job certificat 117066939109. Le résultat dépend du serveur, de la date et des capacités du client ; il est daté et ne préjuge pas des renouvellements futurs.

Ne pas confondre certificat et chiffrement TLS

La signature du certificat est produite par l'autorité émettrice. Le bourrage identifié concerne cette signature, et non un chiffrement du message avec le format 00 02 étudié dans le TP.

La connexion a négocié TLSv1.3 avec TLS_AES_128_GCM_SHA256. Le trafic est protégé par un chiffrement symétrique authentifié. TLS 1.3 n'utilise pas l'ancien échange de clés RSA. Le certificat seul ne donne ni le groupe d'échange éphémère négocié, ni le schéma de signature de CertificateVerify. L'outil ne prétend pas avoir observé ces paramètres.

Validation et réponses de compréhension

Tests reproductibles

La commande python Walter_Arthur_main.py --test exécute quinze tests. Le journal local tests_locaux.txt conserve leur résultat. La première exécution GitHub Actions a passé les mêmes tests et les démonstrations sous Python 3.10 et 3.13.

Partie	Contrôles principaux
1	300 exponentiations, inverses, UTF-8, vide, zéros initiaux, limites et signatures altérées
2	Comptage exact des bits, total identique sur plusieurs messages, exemple CRT officiel et comparaisons sur les deux clés
3	Structure du padding, rejets, messages longs, emoji à cheval sur deux blocs, aléa et altérations
Certificat	Trois tests locaux RSA, EC et clé EC avec signature RSA de l'émetteur

Les trois tests X.509 nécessitent cryptography et s'exécutent avec python outils/test_certificat.py. Ils utilisent des certificats locaux de test générés pour vérifier le lecteur ; ces certificats ne servent jamais de preuve pour les valeurs Springer.

Pourquoi utiliser deux paires de clés

La clé d'Alice est utilisée pour la confidentialité : Bob chiffre pour elle. La clé de Bob sert à la signature : Alice vérifie ce que Bob a signé. Chiffrer et signer remplissent deux objectifs différents.

Pourquoi signer une empreinte

Une empreinte a une taille fixe quel que soit le message. Elle permet de comparer le contenu avant et après transmission. Dans ce TP, la signature applique l'opération privée de Bob aux blocs de l'empreinte ; un hash seul ne permettrait pas d'attribuer le message à Bob.

Le terme HMAC présent dans l'avertissement du sujet ne décrit pas cette construction : un HMAC repose sur une clé secrète partagée. Notre échange utilise une signature asymétrique simplifiée.

Pourquoi vérifier toute la restitution

Le fait de retrouver un texte lisible ne prouve pas à lui seul son intégrité. La signature est vérifiée sur le texte complet après concaténation. Les tests de modification montrent le rejet de cas précis ; ils ne constituent pas une preuve de résistance générale aux falsifications.

Les tests chronométriques ne fixent aucun seuil de performance, car un système chargé pourrait changer les durées sans modifier la correction mathématique.

Livrables limites et références

Correspondance avec le sujet

Consigne	Réalisation
Partie 1 fonctions et échange	Walter_Arthur_util.py et partie1 dans main
Partie 1 SHA-256 et limite	Empreinte entière en deux blocs et limite de 25 octets
Partie 2 traces	home_mod_expnoent_trace et interprétation page 7
Partie 2 CRT	home_preparer_crt et home_rsa_crt
Bonus temporel et risque CRT	Mesures locales et discussion page 9
Partie 3 bourrage	home_bourrer et home_debourrer
Partie 3 échange long	home_envoyer_blocs et home_recevoir_blocs
Partie 3 certificat	PEM, JSON daté et analyse page 12

Limites et choix d'adaptation

Les clés sont celles du professeur. Le code n'ajoute ni RSA-PSS, ni RSA-OAEP, ni gestion de confiance des clés de Bob. La signature en deux blocs conserve le SHA-256 avec ces petites clés, mais n'est pas un schéma standard : les deux blocs sont signés indépendamment. La construction est conservée pour les objectifs pédagogiques et ne doit pas être présentée comme sécurisée.

Le CRT exige deux premiers distincts et des paramètres cohérents. Le padding est validé avec une erreur générique mais sans garantie de temps constant. La taille totale du texte de la partie 3 dépend de la mémoire disponible et le nombre de blocs révèle sa longueur approximative.

Le remplacement de MD5 ne change pas la capacité du module. Les résultats de temps sont propres à une exécution. Le certificat observé pourra être renouvelé ; le PEM et la date permettent de conserver la preuve étudiée.

Fichiers à déposer et vérification

Déposer Walter_Arthur_main.py, Walter_Arthur_util.py et docs/Walter_Arthur.pdf dans le devoir Teams. Les tests, les résultats JSON et la version Word sont disponibles dans le dépôt pour expliquer et reproduire le travail. Après dépôt, contrôler les pièces jointes et la confirmation de remise dans Teams.

Références

Sources du TP : TP_RSA_Enonce.pdf pages 2 à 4 ; Exp_CRT_RSA.pdf pages 1 et 2 ; RSA_B_A.py, Abdeljalil ABBAS-TURKI. Les copies originales sont conservées dans TP_secu_1/sources.

Compléments techniques : RFC 8017, sections 7.2 et 8.2, https://www.rfc-editor.org/rfc/rfc8017.html ; RFC 8446, TLS 1.3, https://www.rfc-editor.org/rfc/rfc8446.html. Documentation VS Code : https://code.visualstudio.com/docs/python/run. Sources consultées le 16 octobre 2026.

Preuve du relevé : https://github.com/zeylon-labs/master2-TP/actions/runs/38928738912. Démonstrations et mesures locales : resultats/execution_complete.json et resultats/message_long.json.